



Neural Networks, and Matrix Shapes

1. Core Idea

- Every neuron computes:
 $z = w \cdot x + b$
(dot product of weights and inputs + bias).
 - Confusion comes from notation flips:
 - Math textbooks vs programming libraries (row vs column convention).
 - Regression (1 neuron) vs NN layer (many neurons).
-

2. Terminology

- **Sample** = one data point (row in dataset).
 - **Feature** = one input variable (column in dataset).
 - **Neuron** = one unit in the current layer.
 - **Pre-activation (Z)** = raw weighted sum before activation.
 - **Activation (A)** = output of the layer after applying activation function.
-

3. Regression Setup (special case of NN with 1 neuron)

$$y = Xw$$

- $X \in \mathbb{R}^{(n_{\text{samples}} \times n_{\text{features}})}$
- $w \in \mathbb{R}^{(n_{\text{features}} \times 1)}$
- $y \in \mathbb{R}^{(n_{\text{samples}} \times 1)}$

- ✓ Each sample (row of X) is dotted with the weight vector w.
 - ✓ Produces one prediction per sample.
-

4. Neural Network Layer

$$\mathbf{Z} = \mathbf{XW} + \mathbf{b}$$

$$\mathbf{A} = \sigma(\mathbf{Z})$$

- $\mathbf{X} \in \mathbb{R}^{(n_{\text{samples}} \times n_{\text{inputs}})}$
- $\mathbf{W} \in \mathbb{R}^{(n_{\text{inputs}} \times n_{\text{neurons}})}$
- $\mathbf{b} \in \mathbb{R}^{(n_{\text{neurons}})}$
- $\mathbf{Z}, \mathbf{A} \in \mathbb{R}^{(n_{\text{samples}} \times n_{\text{neurons}})}$

👉 Each row of $\mathbf{Z}$ = values of all neurons for one sample.

👉 Each column of $\mathbf{Z}$ = one neuron's values across all samples.

5. Single Sample vs Many Samples

- **One sample (row vector):**
 $\mathbf{X}: (1 \times n_{\text{inputs}}) \rightarrow \mathbf{Z}: (1 \times n_{\text{neurons}})$ (row vector).
- **Batch of samples:**
 $\mathbf{X}: (n_{\text{samples}} \times n_{\text{inputs}}) \rightarrow \mathbf{Z}: (n_{\text{samples}} \times n_{\text{neurons}})$.

✅ Same formula works — one row = one sample.

6. Convention Flip

- **Math-classroom (your image):**
 $z = \mathbf{W} \cdot \mathbf{a} + b$, where $\mathbf{W}: (n_{\text{neurons}} \times n_{\text{inputs}})$.
- **Programming (NumPy / PyTorch):**
 $\mathbf{Z} = \mathbf{XW} + \mathbf{b}$, where $\mathbf{W}: (n_{\text{inputs}} \times n_{\text{neurons}})$.

✅ Both describe the same dot products, just transposed.

7. Forward Pass Through the Network

For an L-layer neural network (ignoring biases for simplicity):

1. **Layer 1:**
 $\mathbf{Z}[1] = \mathbf{XW}[1] + \mathbf{b}[1], \quad \mathbf{A}[1] = \sigma(\mathbf{Z}[1])$
2. **Layer 2:**
 $\mathbf{Z}[2] = \mathbf{A}[1]\mathbf{W}[2] + \mathbf{b}[2], \quad \mathbf{A}[2] = \sigma(\mathbf{Z}[2])$

3. **Output Layer (regression case):**
 $Z[L] = A[L-1]W[L] + b[L]$, $A[L] = Z[L]$
(identity activation for regression).
-

8. Intuition for Z and A

- $Z[l]$ = the **raw input into neurons** of layer l (pre-activation).
- $A[l]$ = the **output values** after activation.

Flow:

$A[0] = X \rightarrow (Z[1], A[1]) \rightarrow (Z[2], A[2]) \rightarrow \dots \rightarrow A[L]$

- ✓ Z feeds into activation $\rightarrow A$.
 - ✓ A becomes the input for the next layer.
-

9. Dot Product Cheat Section

Weighted Sum View

$$x \cdot w = \sum (x_i w_i)$$

- Each feature multiplied by its weight, summed.
 - **Neuron computation perspective.**
-

Alignment View

$$x \cdot w = \|x\| \|w\| \cos(\theta)$$

- Geometric meaning = how much two vectors point in the same direction.
 - Neuron = detector of alignment with its weight vector.
-

Connection

- Same operation.
- **Weighted sum** = practical view for coding.

- **Alignment** = geometric intuition for learning/classification.

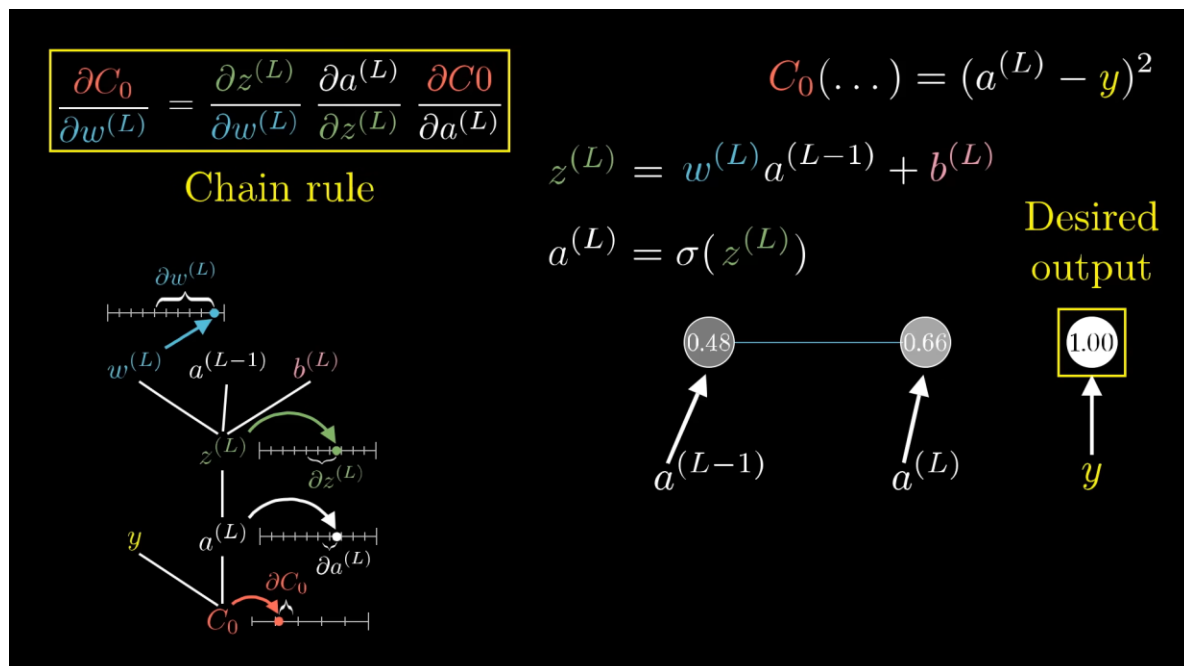
10. Practical Rule (never get lost again)

- Always stick to **programming convention**:
 $Z = XW + b$
- Rows of X = samples.
- Columns of X = features.
- Columns of W = neurons.
- Each row of Z = all neuron values for one sample.

⚡ Ultimate TL;DR

- **Regression** = 1-neuron network.
- **Neural net layer** = many neurons at once.
- Z = pre-activation (raw weighted sum).
- A = activation output (input to next layer).
- **Dot product** = weighted sum *and* alignment.
- Stick with $Z = XW + b \rightarrow$ works for one sample or many.

Backpropagation calculus: <https://www.youtube.com/watch?v=tleHLnjs5U8>



Here we see how a change in $w^{(L)}$ causes a change in the Cost Function and how it is calculated.

$$\frac{\partial C_0}{\partial w^{(L)}} = \frac{\partial z^{(L)}}{\partial w^{(L)}} \frac{\partial a^{(L)}}{\partial z^{(L)}} \frac{\partial C_0}{\partial a^{(L)}}$$

$$\frac{\partial C_0}{\partial a^{(L)}} = 2(a^{(L)} - y)$$

$$\frac{\partial a^{(L)}}{\partial z^{(L)}} = \sigma'(z^{(L)})$$

$$\frac{\partial z^{(L)}}{\partial w^{(L)}} = a^{(L-1)}$$

$$C_0 = (a^{(L)} - y)^2$$

$$z^{(L)} = w^{(L)} a^{(L-1)} + b^{(L)}$$

$$a^{(L)} = \sigma(z^{(L)})$$

This $w^{(L)}$ is not the only part of the gradient calculation, we want to also calculate how a change in biases or activations changes it.

Backpropagation calculus | Deep Learning Chapter 4

$$\frac{\partial C_0}{\partial w^{(L)}} = \frac{\partial z^{(L)}}{\partial w^{(L)}} \frac{\partial a^{(L)}}{\partial z^{(L)}} \frac{\partial C_0}{\partial a^{(L)}} = a^{(L-1)} \sigma'(z^{(L)}) 2(a^{(L)} - y)$$

$$\nabla C = \begin{bmatrix} \frac{\partial C}{\partial w^{(1)}} \\ \frac{\partial C}{\partial b^{(1)}} \\ \vdots \\ \frac{\partial C}{\partial w^{(L)}} \\ \frac{\partial C}{\partial b^{(L)}} \end{bmatrix}$$

$$C_0 = (a^{(L)} - y)^2$$

$$z^{(L)} = w^{(L)} a^{(L-1)} + b^{(L)}$$

$$a^{(L)} = \sigma(z^{(L)})$$

5:34 / 10:17 • What do the derivatives mean? >

For a bias:

Backpropagation calculus | Deep Learning Chapter 4

$$\frac{\partial C_0}{\partial b^{(L)}} = \frac{\partial z^{(L)}}{\partial b^{(L)}} \frac{\partial a^{(L)}}{\partial z^{(L)}} \frac{\partial C_0}{\partial a^{(L)}} = 1 \sigma'(z^{(L)}) 2(a^{(L)} - y)$$

Diagram illustrating the forward pass for bias calculation:

- Inputs: $w^{(L)}$, $a^{(L-1)}$, $b^{(L)}$
- Intermediate value: $z^{(L)}$
- Activation function: $a^{(L)} = \sigma(z^{(L)})$
- Loss function: $C_0 = (a^{(L)} - y)^2$

Diagram illustrating the backward pass for bias calculation:

- Inputs: $a^{(L-1)}$ (0.48), $a^{(L)}$ (0.66), y (1.00)
- Intermediate value: $z^{(L)}$
- Output: C_0

6:05 / 10:17 · Sensitivity to weights/biases >

For the activation:

Backpropagation calculus | Deep Learning Chapter 4

$$\frac{\partial C_0}{\partial a^{(L-1)}} = \frac{\partial z^{(L)}}{\partial a^{(L-1)}} \frac{\partial a^{(L)}}{\partial z^{(L)}} \frac{\partial C_0}{\partial a^{(L)}} = w^{(L)} \sigma'(z^{(L)}) 2(a^{(L)} - y)$$

Diagram illustrating the forward pass for activation calculation:

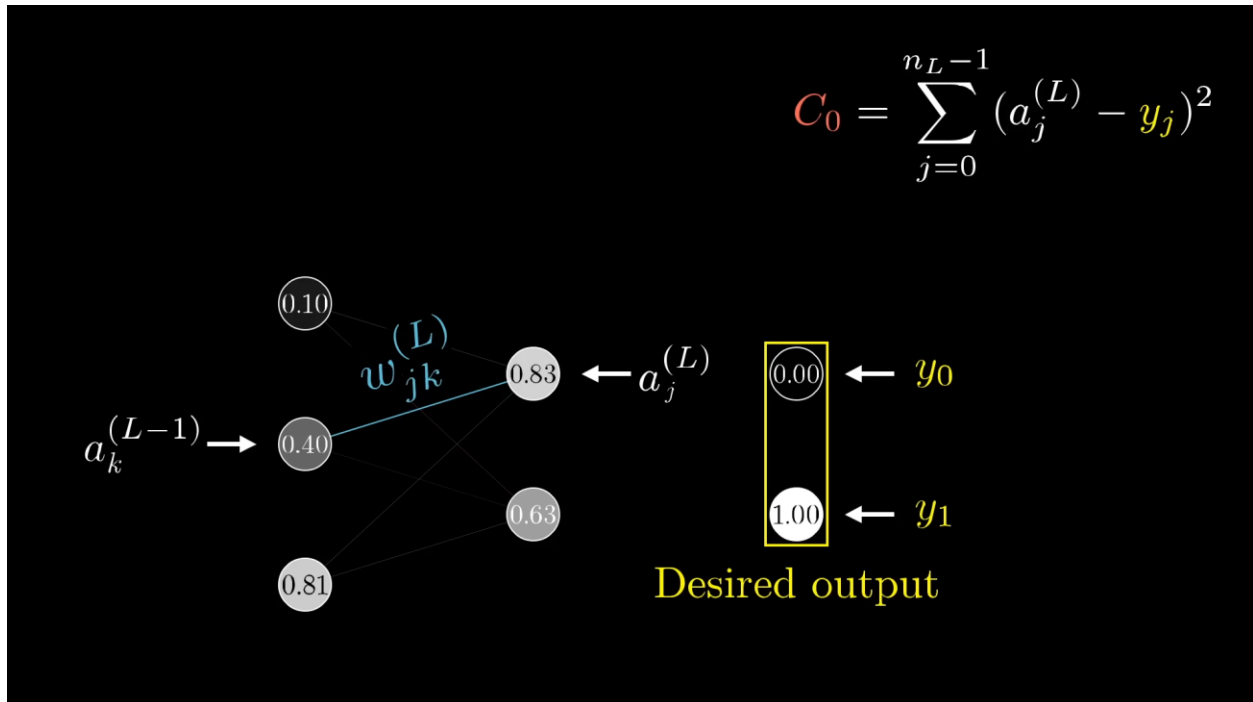
- Inputs: $w^{(L)}$, $a^{(L-1)}$, $b^{(L)}$
- Intermediate value: $z^{(L)}$
- Activation function: $a^{(L)} = \sigma(z^{(L)})$
- Loss function: $C_0 = (a^{(L)} - y)^2$

Diagram illustrating the backward pass for activation calculation:

- Inputs: $a^{(L-1)}$ (0.48), $a^{(L)}$ (0.66), y (1.00)
- Intermediate value: $z^{(L)}$
- Output: C_0

6:27 / 10:17 · Sensitivity to weights/biases >

For non single neurons:



Backpropagation calculus | Deep Learning Chapter 4

$$z_j^{(L)} = w_{j0}^{(L)} a_0^{(L-1)} + w_{j1}^{(L)} a_1^{(L-1)} + w_{j2}^{(L)} a_2^{(L-1)} + b_j^{(L)}$$

$$a_j^{(L)} = \sigma(z_j^{(L)})$$

$$C_0 = \sum_{j=0}^{n_L-1} (a_j^{(L)} - y_j)^2$$

8:05 / 10:17 · Layers with additional neurons >

The idea is the same almost:

Backpropagation calculus | Deep Learning Chapter 4

$$\frac{\partial C_0}{\partial w_{jk}^{(L)}} = \frac{\partial z_j^{(L)}}{\partial w_{jk}^{(L)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \frac{\partial C_0}{\partial a_j^{(L)}}$$

$$z_j^{(L)} = \dots + w_{jk}^{(L)} a_k^{(L-1)} + \dots$$

$$a_j^{(L)} = \sigma(z_j^{(L)})$$

$$C_0 = \sum_{j=0}^{n_L-1} (a_j^{(L)} - y_j)^2$$

8:29 / 10:17 · Layers with additional neurons >

Where we use the chain rule but it's just a bit more complicated since more nodes.

The only change is in the partial derivative of the cost wrt the activation:

Backpropagation calculus | Deep Learning Chapter 4

$$\frac{\partial C_0}{\partial a_k^{(L-1)}} = \sum_{j=0}^{n_L-1} \frac{\partial z_j^{(L)}}{\partial a_k^{(L-1)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \frac{\partial C_0}{\partial a_j^{(L)}}$$

$$z_j^{(L)} = \dots + w_{jk}^{(L)} a_k^{(L-1)} + \dots$$

$$a_j^{(L)} = \sigma(z_j^{(L)})$$

$$C_0 = \sum_{j=0}^{n_L-1} (a_j^{(L)} - y_j)^2$$

8:42 / 10:17 · Layers with additional neurons >

The difference is that the activation influences not only one single neuron but all neurons in the next layer. So you sum over the layer L!